

树与二叉树

层次关系怎样递归表示、遍历并保持结构不变量

408 · 数据结构 Topic DS-04

母问题：树把层次关系存成什么，怎样遍历、恢复和变换，哪些信息绝不能弄丢？

本 Topic 实际处理两类问题。第一类是普通树结构：

父子层次 → 存储表示 → 递归拆成子树 → 遍历/转换 → 检查结构是否仍正确。

遍历、重建、计数、孩子兄弟转换和线索化都在问同一件事：**当前表示保存了哪些关系，这一步操作之后还能可靠推出什么。**第二类是 Huffman 编码：

字符权值 → 叶节点深度 → 带权路径长度 WPL → 反复合并最小权值 → 前缀码。

它也使用树，但目标不同：前一类关心结构关系是否被正确保存和读取；Huffman 关心怎样安排叶节点深度，使加权后的总代价最小。因此两类问题可以共用“路径、分支、叶节点”等树语言，却不能用同一套做题目标强行统一。

本册定位与 Owner 边界

- **Owns:** 树/森林与二叉树的性质和存储、孩子兄弟转换、递归与迭代遍历、线索化、高度、Huffman 主干。
- **Uses:** DS02 的栈/队列端点契约；DS05 的优先队列接口。
- **Stops:** 堆内部上滤下滤、BST/AVL/B/B+ 查找与旋转、图遍历正确性归各自 Topic。

目录

1 对象与表示	3
1.1 先统一度、深度、高度与几类二叉树	3
2 遍历：先分 DFS/BFS，再问“根在什么时候被访问”	3
3 遍历序列与结构信息：为什么有些组合能唯一恢复？	4
3.1 含中序的恢复：根定位 + 左右切分	4
3.2 前序 + 后序：丢失的是单孩子方向	4
3.3 用前序、后序的位置判断祖先关系	5
3.4 遍历序列的相等、逆序与相对次序反转	5
3.5 附加形状约束可以补回缺失信息	5
4 树与森林的表示转换、计数与存储	6
4.1 孩子兄弟表示：L/R 怎么翻译，数量怎么数	6
4.2 必然会复原的计数不变量	6
4.3 层容量：向下看最多能放多少，向上数需要多少父节点支撑	7

4.4	存储表示：根据常用操作选择结构	7
5	遍历应用与结构操作：题目目标决定什么时候处理根、递归返回什么	8
5.1	层序的最小深度：首个叶节点即可停止	8
5.2	序列化与反序列化：空子树也是信息	8
5.3	后序递归：每棵子树只返回父节点真正需要的信息	9
6	线索二叉树：把“结构边”与“遍历邻接”装进同一组指针	9
6.1	线索化的统一写法：pre 始终指向刚访问的节点	9
6.2	为什么中序能双向走，而先序/后序只有一个方向天然容易	10
6.3	首尾边界与“线索化后还剩几个空链域”	10
6.4	带头节点的中序线索树	10
7	Huffman：用树深表示编码代价，再让总代价最小	10
7.1	结构不变量与三大关键推论	10
7.2	贪心最优性与唯一性判据	11
7.3	m 叉 Huffman：先补齐叶数同余，再每轮合并 m 个最小值	11
7.4	前缀码容量：一个短码占用一整棵后缀子树	11
8	代码：从节点所有权到访问顺序	12
8.1	节点与递归遍历	12
8.2	显式栈与层序队列	13
8.3	Huffman 合并调用	14
8.4	测试证据	15
9	复杂度与概念边界	16
10	遇到题目时从哪里开始	16

1 对象与表示

二叉树节点至少包含值、左子树、右子树。子树为空是合法状态，不是异常；叶节点的两个孩子都为空。用唯一所有权表示时，父节点拥有子节点，销毁根即可递归释放整棵树。树的结构不变量是：根没有父节点，其余节点恰有一个父节点；沿孩子指针不会回到祖先；从根遍历时每个可达节点只应被访问一次。

1.1 先统一度、深度、高度与几类二叉树

树是一个连通、无环的层次结构；根节点没有双亲，其余节点恰有一个双亲。节点的度是孩子数，树的度是所有节点度的最大值；叶节点度为零。深度通常按“从根到该节点经过的边数”计，树高取最大深度；若教材改用“根为第 1 层”的层数口径，就要整体换算，不能混用。

二叉树不是“每层最多两个节点”，而是每个节点都有有序的左、右子树，空子树也属于结构的一部分。常见附加约束也要分开：**严格/正则二叉树**要求每个节点度只能为 0 或 2；**满二叉树**（perfect binary tree）要求每一层全部填满；**完全二叉树**只要求除最后一层外都满、最后一层从左到右连续。平衡二叉树限制高度差，二叉排序树增加关键字次序，线索二叉树则重新利用空指针保存遍历邻接。名称相近，但约束不同，不能互相代用。

同一棵树四种读法

对根 1、左子树 2(4,5)、右子树 3，先序（前序）为 1,2,4,5,3，中序为 4,2,5,1,3，后序为 4,5,2,3,1，层序为 1,2,3,4,5。树本身没有变化：先/中/后序只改变 DFS 中“什么时候记录根”，层序则改用队列按层取节点。本文把“先序”和“前序”视为同义词。

2 遍历：先分 DFS/BFS，再问“根在什么时候被访问”

对一棵二叉树，先序、中序、后序不是三套互不相关的“走法”，而是同一次深度优先搜索（DFS）在三个事件点记录根节点。对当前节点 p ，一次递归调用天然经历

进入 p $\rightarrow$ 左子树返回 $\rightarrow$ 右子树返回。

若分别在这三个时刻记录 p ，就得到先序、中序、后序：

遍历	访问时机	递归次序
先序	进入节点时	NLR
中序	左子树返回后	LNR
后序	右子树也返回后	LRN

因此三者都属于 DFS；所谓“前/中/后”描述的是根 N 相对左右子树 L, R 的记录时机，而不是三种不同搜索方向。

层序遍历则属于**广度优先搜索（BFS）**：把已经发现、但还没有展开的节点放进 FIFO 队列，先处理浅层，再处理深层。程序真正需要确定三件事：

待处理节点放在哪里 + 下一个取谁 + 什么时候把节点写入结果。

这个“待处理节点集合”也常称为 **frontier**（搜索边界）。DFS 用递归调用栈或显式栈保存它，BFS 用队列保存它。

迭代实现时真正要守的三个不变量

1. **覆盖**：每个真实节点恰好进入待处理结构一次；空指针不入栈、不入队。
2. **次序**：栈是 LIFO，所以先序必须先压右、再压左；中序必须先沿左链压到底；层序必须按“左孩子先入队、右孩子后入队”的父层顺序扩展。
3. **完成状态**：后序要求知道“右子树是否已经完成”，因此单栈实现通常需要 `lastVisited`；双栈实现用额外 $O(n)$ 空间换取简单控制。

所有普通遍历都访问每个节点常数次，时间为 $O(n)$ 。递归 DFS 与单栈 DFS 的辅助空间最坏为 $O(h)$ ，其中 h 是树高；双栈后序最坏为 $O(n)$ ；BFS 队列最坏保存某一层的节点，辅助空间为 $O(w)$ ，其中 w 是最大宽度。链状树上 DFS 栈可能达到 $O(n)$ 而 BFS 近似常数空间；接近完全的树上则 BFS 队列可能达到 $\Theta(n)$ 。

3 遍历序列与结构信息：为什么有些组合能唯一恢复？

遍历把一棵有序二叉树压缩成线性序列。反向恢复结构时，核心不是“给了几条序列”，而是这组信息能否同时确定：

当前子树的根是谁 + 左右子树的边界在哪里。

以下结论默认每个节点都能被唯一识别。若节点值允许重复，仅凭“值”在中序中会出现多个候选位置，此时还需要节点编号、出现序号或题目另给的区分规则。

3.1 含中序的恢复：根定位 + 左右切分

对任一当前子树，若根为 r ，则

$$\text{Inorder}(T) = \text{Inorder}(T_L) \ r \ \text{Inorder}(T_R).$$

因此一旦知道 r ，中序中的位置就唯一确定左、右子树的节点集合。先序的首元素、后序的末元素都直接给出当前根；层序中属于当前子树且最先出现的节点也必为当前根。于是“先序 + 中序”“后序 + 中序”“层序 + 中序”在节点身份唯一时都可以递归唯一恢复。

程序构造时若反复线性扫描中序定位根，最坏可能退化为 $O(n^2)$ ；先建立“节点身份 $\mapsto$ 中序下标”的映射，先序 + 中序或后序 + 中序即可在 $O(n)$ 时间内完成构造，递归栈为 $O(h)$ 。

3.2 前序 + 后序：丢失的是单孩子方向

前序首元素与后序末元素都给出根。若当前根有两个非空孩子，前序中根后的第一个节点给出左子树根，它在后序中的结束位置可以确定左子树规模，剩余节点只能属于右子树，因此两棵孩子子树的边界仍可确定。真正的信息丢失只发生在当前根只有一个非空孩子时：把这棵唯一孩子子树放在左边或右边，都不会改变前序和后序。

所以对一对合法、节点身份唯一的前序/后序序列，若兼容树中有 k 个单孩子节点，则每个节点都独立贡献一次“左/右”二元选择，兼容的有序二叉树共有

$$2^k$$

棵；唯一恢复当且仅当 $k = 0$ 。这个条件应表述为**不存在度为 1 的节点**，即每个非叶节点恰有两个孩子；它比“每一层都填满”的满二叉树条件更弱。

3.3 用前序、后序的位置判断祖先关系

对不同节点 x, y , 记 $pre(x)$ 、 $post(x)$ 分别为它们在先序、后序中的位置, 则

$$x \text{ 是 } y \text{ 的祖先} \iff pre(x) < pre(y) \wedge post(x) > post(y).$$

这是 DFS 的进入/完成区间关系: 祖先先进入、后完成。它只能给出祖先-后代或不同分支关系, 不能直接把“祖先”强化成“父亲”, 也不能把“不同分支”强化成“兄弟”。父子和兄弟还需要直接父节点这一层额外结构信息。

还有一个容易被忽略、但可以直接用于判断题的次序结论: **只抽取叶节点后, 先序、中序、后序中的叶节点相对顺序完全相同**。对任意两个不同叶节点 u, v , 它们互不可能为祖先; 令 $a = LCA(u, v)$, 则二者分别落在 a 的左右两棵不同子树中。三种 DFS 都会先完整处理左子树, 再处理右子树, 因此 u, v 的先后只由它们位于 a 的哪一侧决定, 不受“根在何时记录”影响。这个结论只比较叶节点之间的相对次序, 不能推广为三种完整遍历序列相同。

3.4 遍历序列的相等、逆序与相对次序反转

下面的形状结论默认节点身份唯一; 若只比较可重复的节点值, 序列偶然相等不能推出同样的结构限制。对任一非空子树, 先序和中序分别是 NLR 与 LNR , 因此二者完全相同当且仅当所有节点的左子树都为空; 同理, 中序和后序完全相同当且仅当所有节点的右子树都为空。先序与后序若完全相同, 则除空树外只能有一个节点, 因为根必须同时位于序列首尾。

更有用的是

$$Pre(T) = reverse(Post(T)) \iff T \text{ 中每个节点至多有一个非空孩子}$$

也就是整棵树没有分叉。原因是后序倒置的局部顺序为 NRL ; 若某节点同时有左右两棵非空子树, 先序要求整块左子树先于右子树, 而后序倒置要求整块右子树先于左子树, 不可能相同。若每个节点只有一棵非空孩子子树, 则 NLR 与 NRL 在每层都退化成“根 + 唯一子树”, 两序列正好互逆。

中序还可以和另一条 DFS 序列组成**相对次序反转判据**。若

$$pre(x) < pre(y), \quad in(x) > in(y),$$

则 x 必是 y 的祖先, 且 y 位于 x 的左子树; 若二者互不为祖先, 它们在最近公共祖先处属于左右不同分支, 前序和中序的相对次序不会反转。对后序有对偶结论: 若

$$post(x) < post(y), \quad in(x) > in(y),$$

则 y 必是 x 的祖先, 且 x 位于 y 的右子树。相对次序没有反转时通常不能单独推出祖先关系。

3.5 附加形状约束可以补回缺失信息

完全二叉树的节点数 n 一旦确定, 空槽位置和整棵树的形状已经唯一确定; 此时单独一条先序、中序、后序或层序序列都只是在固定形状上依次赋值, 因此足以恢复带值结构。类似地, BST 的关键字有序约束也可能让单个先序/后序足以恢复, 但该有序索引机制由 DS09 Own, 本册只保留这一接口边界。

“序列是否存在对应树”和“若存在是否唯一”是两个不同问题。长度、节点身份集合、根位置以及递归划分必须彼此相容; 若某一步无法形成同一节点集合, 则不存在对应二叉树, 而不是“存在多棵”。

4 树与森林的表示转换、计数与存储

一般树的节点可以有任意多个孩子。孩子兄弟表示只保留两个方向：

firstChild = 第一个孩子, nextSibling = 下一个兄弟.

把 firstChild 解释为二叉树左孩子、 nextSibling 解释为右孩子, 就得到树到二叉树的转换。森林则把各棵树的根按兄弟链连接: 第一棵树根是二叉树根, 后续树根依次位于右链。

该转换保持层次与兄弟次序, 不保持“二叉树左右孩子”的原语义。由指针解释可推出遍历对应:

树的先根遍历 = 对应二叉树的先序遍历, 树的后根遍历 = 对应二叉树的中序遍历.

森林遍历还要沿根的兄弟链完成每棵树, 不能只处理第一棵。

4.1 孩子兄弟表示: L/R 怎么翻译, 数量怎么数

把转换后二叉树中的左、右指针记作 L, R , 它们必须翻译成两种不同动作:

L = 进入第一个孩子, R = 转到下一个兄弟.

因此在原树中, 节点 y 是 x 的某个孩子, 当且仅当在二叉表示中可以从 x 沿一条 LR^k 路径到达 y ; R 本身不下降层级, 只在同一父节点的孩子链中向右移动。森林各棵树的根同样构成一条 R 链, 所以二叉表示根的右子树恰好包含第 2 棵及以后所有树, 而根的左子树恰好包含第 1 棵树除根以外的全部节点。

这种翻译还能直接给出几个计数不变量。设森林共有 N 个节点, 其中原树/森林叶节点数为 L_0 。原节点没有孩子当且仅当其 firstChild 为空, 所以转换后二叉树中

$$\#(\text{left} = \emptyset) = L_0.$$

转换结果仍是一棵含 N 个节点的二叉树, 左右孩子空槽总数恒为 $N + 1$, 于是

$$\#(\text{right} = \emptyset) = N + 1 - L_0.$$

若用定长 m 个孩子指针存原 m 叉树, 原表示有 mN 个孩子槽, 而孩子兄弟表示只需 $2N$ 个槽, 因此只比较孩子指针存储时节省 $(m - 2)N$ 个指针域。这个结论不包含额外 `tag`、`parent` 或对齐开销。

4.2 必然会复原的计数不变量

任意有限非空有根树若用 n_i 表示度为 i 的节点数, 则边数一方面等于非根节点数 $n - 1$, 另一方面等于所有孩子数之和, 因此

$$\sum_{i \geq 0} i n_i = n - 1.$$

配合 $n = \sum_i n_i$ 可直接得到叶节点的一般式

$$n_0 = 1 + \sum_{i \geq 1} (i - 1) n_i.$$

它说明度为 1 的节点不改变叶子盈余, 度为 $i \geq 2$ 的节点相对一条继续向下的主干多贡献 $i - 1$ 个叶子分支。

对二叉树进一步只保留 $i = 0, 1, 2$ 。设 n_0 个叶节点、 n_1 个度为 1 的节点、 n_2 个度为 2 的节点。按孩子数计数, 边数为 $n_1 + 2n_2$, 从而

$$n_0 + n_1 + n_2 - 1 = n_1 + 2n_2 \implies n_0 = n_2 + 1.$$

因此严格/正则二叉树（没有度为 1 节点）有 $n = 2n_0 - 1$ 个节点；若题目给出叶数或双分支节点数，可以先用这个关系排除不可能的选项。高度为 h （根深度为 0）时，节点数满足 $h + 1 \leq n \leq 2^{h+1} - 1$ ；反过来，给定 n ，最小高度为 $\lceil \log_2(n+1) \rceil - 1$ ，最大高度为 $n - 1$ 。若教材把空树高度定义为 -1 ，上述递归式应写成 $H(\emptyset) = -1$ 、 $H(T) = 1 + \max(H_L, H_R)$ ；若定义为空树 0，则所有高度整体平移一位，不能混用。

4.3 层容量：向下看最多能放多少，向上数需要多少父节点支撑

若根记为第 1 层，每个节点至多有 d 个孩子，则第 i 层节点数 N_i 满足

$$N_i \leq d^{i-1},$$

高度（层数）为 h 时总节点数满足

$$N \leq 1 + d + \cdots + d^{h-1} = \frac{d^h - 1}{d - 1} \quad (d > 1).$$

这两个式子描述的是**分支上限给出的容量上界**。若题面说“树的度恰为 d ”，还额外要求至少有一个节点真的拥有 d 个孩子；当 $h \geq 2$ 时，在保持高度为 h 的同时，最少只需在一条最长根叶路径上的某个非叶节点扩出额外 $d - 1$ 个叶子，因此有

$$N_{\min} = h + d - 1.$$

这里的 h 按层数计；若根深度为 0，只需用“层数 = 最大深度 + 1”换算，不得把两套口径混在同一公式中。

容量题还经常反过来问：“总共只有 n 个节点，第 k 层最多能放多少？”这时不能只写 d^{k-1} ，因为第 k 层的节点必须由前 $k - 1$ 层祖先支撑。若第 k 层想放 x 个节点，则至少需要

$$M_d(x, k) = \sum_{j=0}^{k-1} \left\lceil \frac{x}{d^j} \right\rceil$$

个节点：第 $k - 1$ 层至少要有 $\lceil x/d \rceil$ 个父节点，再逐层向上取整，直到根。于是 x 可行的必要充分条件是

$$x \leq d^{k-1} \quad \text{且} \quad M_d(x, k) \leq n.$$

所以这类题的固定做法是：先假设目标层有 x 个节点，再逐层向上取整算出最少需要多少父节点，最后检查总节点数够不够。二叉树只需令 $d = 2$ 。

完全二叉树按层序从 1 开始编号时，节点 i 的孩子是 $2i$ 、 $2i + 1$ （越界即不存在），父节点为 $\lfloor i/2 \rfloor$ 。因此最后一个非叶节点编号是 $\lfloor n/2 \rfloor$ ，叶节点恰为 $\lfloor n/2 \rfloor + 1, \dots, n$ ；判断一棵按数组给出的树是否完全，可检查所有非空节点是否已经出现“右孩子而无左孩子”，以及从第一次空槽之后是否仍有非空槽。若采用 0 下标，公式整体改为左孩子 $2i + 1$ 、右孩子 $2i + 2$ ，考场先写下标约定再代入。

4.4 存储表示：根据常用操作选择结构

顺序存储适合完全二叉树或形状已经固定的树：数组下标能直接给出父子关系，但稀疏树会留下大量空位置。链式表示中的空指针可以直接按“总指针字段数 - 非空指针字段数”计算。若每个节点固定提供 c 个孩子指针域，则 n 个节点共有 cn 个孩子槽位，而任意非空树只有 $n - 1$ 条父子边，所以非空孩子指针恰有 $n - 1$ 个，空孩子指针数为

$$cn - (n - 1) = (c - 1)n + 1.$$

二叉链表取 $c = 2$ ，立即得到 $n + 1$ 个空孩子字段，这正是线索化可利用的空间；三叉树若每个节点固定保留三个孩子指针，则得到 $2n + 1$ 个空孩子字段。

若节点还额外保存一个 `parent` 指针，则除根外的 $n - 1$ 个节点的 `parent` 域非空，根的 `parent` 为空。于是“ c 个孩子域 + 1 个 `parent` 域”一共 $(c + 1)n$ 个指针字段：孩子指针中有 $n - 1$ 个非空，`parent` 指针中也有 $n - 1$ 个非空，因此空指针总数为

$$(c + 1)n - 2(n - 1) = (c - 1)n + 2.$$

例如二叉树三叉链表 (`left/right/parent`) 取 $c = 2$ ，空指针总数为 $n + 2$ 。这里必须区分“一个节点有几个字段”“有多少字段指向该节点”和“该节点的度”，三者不是同一个计数对象。

一般树可用双亲表示（每节点一个 `parent` 下标）、孩子链表（每节点挂一条孩子链）或孩子兄弟表示；前者便于向上走，后两者便于枚举孩子。选型时分别问“是否频繁找父亲”“是否频繁插入/删除孩子”“是否需要保留兄弟顺序”，不要因为都是树就默认同一种节点结构。

孩子兄弟转换的复杂度为 $O(n)$ ，逆转换同样为 $O(n)$ 。转换后右指针表示兄弟，不再是普通二叉树里的右孩子。因此纸面推演时必须分清两种指针含义：沿左指针进入第一个孩子，沿右指针转到下一个兄弟。把右链误当成孩子链会导致节点数重复或树高错误。

5 遍历应用与结构操作：题目目标决定什么时候处理根、递归返回什么

前面的 DFS/BFS 模型落到题目时，不要先背代码，而要先问“父节点决策需要哪一侧的信息已经准备好”。先序在进入节点时就做事，适合复制结构、先处理根再向下传递状态；中序在左子树完成后访问根，最典型用途是利用 BST 的有序性；后序等两棵子树都完成才处理根，适合释放整棵树、计算高度/子树大小、验证平衡、直径与自底向上合并。若目标按深度推进、找最近层、统计每层或求宽度，则优先考虑层序 BFS。

迭代实现只是把递归现场显式化：先序栈“弹出-访问-压右-压左”；中序“沿左链压栈-弹出访问-转右”；后序单栈用 `lastVisited` 判断右子树是否完成，或用双栈换取更简单控制。层序遍历用队列逐层展开；若要输出层号，可在队列元素保存 `level`，也可每轮先固定当前队列长度。树没有环时通常不需要 `visited`；一旦节点结构可能共享子树或退化为图，就必须补 `visited`，否则会重复展开。

5.1 层序的最小深度：首个叶节点即可停止

若目标是根到最近叶子的边数，层序遍历按深度递增出队；第一次取到左右孩子均为空的节点时，其层号就是最小深度。空树的返回值必须先约定（常见为 0），根为叶时深度为 1 或 0 取决于题目按节点数还是边数计，不能混写。与求最大深度的 DFS 相比，最小深度的 BFS 可以在首个叶节点停止，但停止前仍要保证本层节点按同一层号处理。

队列元素可保存 (`node, depth`)，或用“本轮队列长度 + 层号”推进。每个节点最多入队一次，时间为 $O(n)$ ，最坏空间为树的最大宽度 $O(w)$ ；若树退化为链，BFS 仍可能只保存常数个节点，而递归 DFS 的栈达到 $O(h)$ 。这个例子说明遍历方式由题目目标决定：找最近层优先 BFS；需要先汇总左右子树结果时，优先后序 DFS。

5.2 序列化与反序列化：空子树也是信息

把树写成字符串或记录流时，目标不是“输出一遍遍历”，而是建立可逆编码。仅有先序值序列通常不能区分“只有左孩子”和“只有右孩子”；若在先序流中为每个空子树写入统一标记（如 #），解码器每读一个普通值就递归构造左右子树，每读一个空标记就返回空指针，于是消费次序与结构生成次序一一对应。含 n 个真实节点的二叉树恰有 $n + 1$ 个空孩子位置，因此完整流有 $2n + 1$ 个记号，这也是检查截断、冗余输入和非法语法的计数证据。后序加空标记也可按相反方向递归解码。

这里还有一个容易过度推广的边界：“遍历值 + 空标记”并不意味着任意遍历都自动可逆。例如中序串 # A # B # 既可以来自根 A、右孩子 B，也可以来自根 B、左孩子 A；原因是中序本身没有告诉解码器哪个普通值是当前根。若要让中序单独承担序列化，还必须加入括号、子树长度等额外语法。

层序编码则让队列中的父节点依次消费两个孩子记号；空记号仍不可在未约定语法时随意删除。可以裁去末尾连续空标记，但编码器与解码器必须共同约定“缺失的尾部孩子均为空”，同时拒绝父节点已经为空却又出现后代值的非法流。重复 key 不影响这种带空标记编码，因为结构由位置语法确定；它只会破坏依赖“在中序中定位唯一根”的重建方法。

遍历序列不等于结构编码

前序、后序或层序只记录真实节点的访问顺序；序列化还必须保存足以恢复空边界的信息。可逆性证明应回答两问：每个编码记号由哪个节点/空槽生成？解码时何时唯一知道一个子树已经结束？

5.3 后序递归：每棵子树只返回父节点真正需要的信息

许多树题表面不同，实际都可以让后序递归只返回“父节点接下来真正需要的那一点信息”。求直径时，左右子树返回高度，当前节点用 $h_L + h_R$ 更新全局最优，再向上返回 $1 + \max(h_L, h_R)$ ；若每个节点都另行扫描子树高度，会把链状树退化为 $O(n^2)$ 。求两个节点的最近公共祖先时，空节点返回空，命中目标返回自身；左右返回都非空则当前根是最低汇合点，只有一侧非空就把该证据向上传递。

原地展开为先序右链时，朴素做法每次把左子树接到右侧后再扫描其最右端，最坏同样为 $O(n^2)$ 。可选择后序返回展开后的尾节点，或按“右—左—根”的逆先序维护上一个已处理节点并重写 right，使每个节点只参与常数连接。共同方法是先把递归函数说清楚：它返回当前子树的什么信息？还会更新哪个全局答案？遇到空树时返回什么基础值？

6 线索二叉树：把“结构边”与“遍历邻接”装进同一组指针

含 n 个节点的二叉链表有 $2n$ 个孩子指针，而非空边只有 $n - 1$ 条，因此有 $n + 1$ 个空指针。线索化不改变树的父子结构，而是把原本为空的槽位重新解释为某一种遍历序列中的前驱/后继邻接。因为同一个字段可能表示真实孩子或线索，必须增加标志：

$$ltag = 0 \Rightarrow lchild \text{ 是左孩子}, \quad ltag = 1 \Rightarrow lchild \text{ 是前驱线索},$$

右侧同理。真正的不变量是：tag 决定字段语义；真实孩子边保持树结构，线索只表示指定遍历的相邻关系。完整线索化时，原来的 $n + 1$ 个空孩子槽都转成 tag=1 的线索字段，因此“线索字段数”恒为 $n + 1$ ；但其中指针值仍为 NULL 的字段只来自遍历序列首尾没有前驱/后继的边界，两者不能混为一个计数。忽略 tag 递归会把序列边误当结构边，造成重复访问甚至循环。

6.1 线索化的统一写法：pre 始终指向刚访问的节点

不论先序、中序还是后序，线索化都可以维护 pre 指向“该遍历下刚刚访问的节点”。访问当前节点 p 时执行两件邻接修复：若 p 的左孩子为空，就让 $p.lchild = pre$ 并置 $ltag = 1$ ；若 pre 存在且它的右孩子为空，就让 $pre.rchild = p$ 并置 $rtag = 1$ 。然后更新 $pre = p$ 。三种线索树的差异不在这两条赋值，而在访问事件发生的时机：先序在进入节点时访问，中序在左子树完成后访问，后序在左右子树都完成后访问。实现时还有一条结构安全条件：一旦空指针被改成线索，后续递归只能沿 tag=0 的真实孩子边继续；否则先序线索化尤其容易把刚写入的前驱线索再次当左孩子递归，造成循环。

6.2 为什么中序能双向走，而先序/后序只有一个方向天然容易

“已经有线索”不等于“任意节点的前驱和后继都能只靠当前节点直接找到”。能否高效定位，取决于目标邻接是否沿当前节点可见的孩子边继续向下，还是需要先知道父节点才能向上回退。

遍历	前驱	后继
先序	若 $ltag = 1$ ，线索直达；否则一般需要 <code>parent</code> 或从根重新定位	若有真实左孩子先去左孩子；否则有真实右孩子去右孩子；两者都无则沿右线索直达
中序	$ltag = 1$ 时线索直达；否则去左子树最右节点	$rtag = 1$ 时线索直达；否则去右子树最左节点
后序	若有真实右孩子，前驱是右孩子；否则若有真实左孩子，前驱是左孩子；叶节点可沿左线索	$rtag = 1$ 时线索直达；否则一般需要 <code>parent</code> 判断当前节点在父节点哪一侧

这个表呈现出一个对称结构：先序天然适合向后走，后序天然适合向前走，中序两个方向都能用“线索或一侧子树的极端节点”完成。困难方向的共同根源是需要从当前子树回到父层，而普通二叉链表没有 `parent` 指针。

6.3 首尾边界与“线索化后还剩几个空链域”

不带头结点时，遍历序列第一个节点没有前驱，最后一个节点没有后继；但只有当对应的原指针槽本来为空时，才会留下一个空线索域。中序最确定：第一个节点一定没有左孩子，最后一个节点一定没有右孩子，所以非空中序线索树恰有两个空线索域。

先序则不同：最后一个访问节点一定是叶节点，因此其空右指针会作为“无后继”保留；第一个节点是根，只有当根本来没有左孩子时，空左指针才会作为“无前驱”保留。因此普通先序线索树可能只剩一个空链域；若根的左子树为空，则恰好剩两个。后序与之对偶：第一个访问节点是叶节点，所以一定留下“无前驱”的左空线索；最后一个节点是根，只有根本来没有右孩子时才额外留下“无后继”的右空线索。

6.4 带头结点的中序线索树

若引入头结点 (Header)，可把边界也收进统一环：头结点的 `lchild` 指向根，中序第一个节点的前驱与最后一个节点的后继都指向头结点，头结点再按项目约定保存首尾连接。这样“无前驱/无后继”不再用空指针表示，遍历停止条件改为回到 Header。

线索树边界

线索只服务于指定遍历次序； $tag=1$ 才表示该字段是序列邻接， $tag=0$ 仍是结构孩子。先序前驱与后序后继通常需要向父层回退，所以没有 `parent` 时不能把“有线索”理解成所有前驱/后继查询都严格 $O(1)$ 。

7 Huffman：用树深表示编码代价，再让总代价最小

给定符号权重集合 $\{w_1, \dots, w_k\}$ ，带权路径长度定义为 $WPL = \sum_{i=1}^k w_i d_i$ 。

7.1 结构不变量与三大关键推论

1. 严格二叉树（正则二叉树）：树中只有度为 0（叶节点，共 k 个）与度为 2（内部合并节点，共 $k - 1$ 个）的节点，不存在度为 1 的节点，总节点数恒为 $2k - 1$ ；

2. **叶节点放置字符**：字符必须且只能放置在叶节点，内部节点仅记录合并权值；这保证了前缀码性质（任一字符编码绝不是另一字符的前缀）；
3. **等价计算公式**：全树各内部节点的合并权值总和，严格等于所有叶节点的带权路径长度总和：

$$\text{WPL} = \sum_{\text{leaves}} w_i d_i = \sum_{\text{internal}} \text{weight}(\text{node})$$

7.2 贪心最优性与唯一性判据

- **贪心选择性质**：二叉 Huffman 每轮把当前两个最小权值合并；它们可以安排在最深层成为兄弟，合并后转化为规模更小的同型子问题；
- **编码不唯一性**：左右子树约定（左 0 右 1 还是左 1 右 0）以及并加权值的合并次序可能生成不同形态的树和不同码字，但给定权值多重集的最小 WPL 保持相同最优值。

7.3 m 叉 Huffman：先补齐叶数同余，再每轮合并 m 个最小值

若 Huffman 树允许每个内部节点恰有 m 个孩子，设叶数为 L 、内部节点数为 I 。按边数计数：一方面边数是 mI ，另一方面是 $L + I - 1$ ，因此

$$L = (m - 1)I + 1, \quad I = \frac{L - 1}{m - 1}.$$

所以完整 m 叉树的叶数必须满足 $L \equiv 1 \pmod{m - 1}$ 。若真实字符数 k 不满足这个同余条件，就补最少个权值为 0 的虚叶，使 $L' = k + z \equiv 1 \pmod{m - 1}$ ，再每轮取当前 m 个最小权值合并。虚叶不贡献真实字符的 WPL，却保证每次合并都生成一个完整的 m 叉内部节点。因而若题目只给真实字符数 k ，补零后的内部节点数还可以直接写成

$$I = \left\lceil \frac{k - 1}{m - 1} \right\rceil.$$

若题目给的是已经包含虚叶在内的总叶数 L' ，则使用精确式 $I = (L' - 1)/(m - 1)$ 。二叉情形 $m = 2$ 时同余条件自动满足，所以不需要补零。

补零还有一个完全等价的实现视角：不显式画虚叶，而让**第一次合并少于 m 个真实权值**，使第一次合并后剩余候选数满足模 $m - 1$ 的完整合并条件。若补了 z 个零权虚叶，则第一次真实合并的元数为 $r = m - z$ ($z = 0$ 时仍取 $r = m$)，之后每轮都取 m 个最小值。两种做法对真实字符的最终深度与 WPL 等价；前者更利于证明完整 m 叉结构，后者更接近某些教材直接画出的“第一层不满”实字符码树。

合并账本仍然成立：一次 m 路合并把所选权值和 s 重新放回候选集合，同时把 s 加入累计代价；最终

$$\text{WPL} = \sum_{\text{all merge rounds}} s.$$

7.4 前缀码容量：一个短码占用一整棵后缀子树

前缀码可以看成 m 叉 trie 的叶节点集合。若统一把最大允许码长记为 H ，一个长度为 ℓ 的码字一旦成为叶节点，它下面所有后缀都不能再作为码字，因此在深度 H 的共同“槽位层”上占用

$$m^{H-\ell}$$

个叶槽。于是已有一组前缀码长度 $\ell_1, \dots, \ell_k$ 时，容量预算满足

$$\sum_i m^{H-\ell_i} \leq m^H,$$

等价于 Kraft 不等式

$$\sum_i m^{-\ell_i} \leq 1.$$

Kraft 不等式描述“这些码长是否存在某组前缀码”的容量条件；若题目已经给出具体码字，还必须用 trie 检查是否真的存在前缀冲突，不能只看长度。若码字正好占据一棵完整 m 叉码树的全部叶节点（包括为 m 叉 Huffman 补入的零权虚叶），则 Kraft 取等号。特别地，常规 $k \geq 2$ 的二叉 Huffman 对一组真实叶权不需要补虚叶，所以完整实字符码字集合必满足 $\sum_i 2^{-\ell_i} = 1$ ；单字符编码还取决于是否允许空码字的约定。

译码时从根出发，读到一个码元就沿对应边走；到达叶节点便输出该字符并回到根。若输入串在中途停在内部节点，或沿不存在的边行走，编码串非法。对具体码字集合，逐个插入 trie：插入过程中若经过已有叶节点，旧码字是新码字前缀；插入结束时若当前节点已有孩子，新码字是旧码字前缀。

这里要分清三层判定。第一层，**前缀码**只要求无码字是另一码字的前缀；第二层，**完整 Huffman 树的叶集合**还要求内部节点满足相应的满分支结构（ m 叉时可以包含零权虚叶）；第三层，**给定权值的 Huffman 最优码**还必须使 WPL 达到最小。因而“没有前缀冲突”只说明编码可即时译码，不能直接证明它就是这组权值的 Huffman 最优码。

还必须区分树的**边深度**与**最终存储码长**。若一棵 m 叉树的每条分支直接用一个 m 进制码元标记，则叶深度 d 就是码长 d ；但若题目把每条 m 叉边再编码成固定 b 位二进制串（例如 4 叉边分别标为‘00/01/10/11’，此时 $b = 2$ ），则真实二进制码长为 bd ，按比特计的加权长度也等于 b 倍的树边 WPL。只有所有分支标签都等长时才能这样整体乘常数；标签变长时必须逐码字统计。

Huffman 码通常不是固定长度码，压缩收益应与固定长度 $\lceil \log_m k \rceil$ 个 m 进制码元的基线比较；若最终落到二进制存储，还要把每个码元实际占用的比特数纳入成本。权值极不均衡时最长码字可能很深，但 WPL 仍由频率加权。

8 代码：从节点所有权到访问顺序

完整实现位于 code/ds04_tree_binary.hpp，测试位于 code/ds04_tree_binary_test.cpp；代码块直接引用真实源文件。

8.1 节点与递归遍历

```

1
2 struct Node {
3     int value;
4     std::unique_ptr<Node> left;
5     std::unique_ptr<Node> right;
6 };
7
8 inline std::unique_ptr<Node> leaf(int value) {
9     return std::make_unique<Node>(Node{value, nullptr, nullptr});
10 }
11
12 inline void preorder_recursive(const Node* root, std::vector<int>& output) {
13     if (root == nullptr) {
14         return;
15     }

```

```

16  output.push_back(root->value);
17  preorder_recursive(root->left.get(), output);
18  preorder_recursive(root->right.get(), output);
19  }
20
21  inline void inorder_recursive(const Node* root, std::vector<int>& output) {
22      if (root == nullptr) {
23          return;
24      }
25      inorder_recursive(root->left.get(), output);
26      output.push_back(root->value);
27      inorder_recursive(root->right.get(), output);
28  }
29
30  inline void postorder_recursive(const Node* root, std::vector<int>& output) {
31      if (root == nullptr) {
32          return;

```

代码清单 1: 唯一所有权节点与递归先中后序

递归基是空指针立即返回；每个非空节点只处理一次，子树调用返回后再按访问时机写入输出。

8.2 显式栈与层序队列

```

1  postorder_recursive(root->left.get(), output);
2  postorder_recursive(root->right.get(), output);
3  output.push_back(root->value);
4  }
5
6  inline std::vector<int> preorder_iterative(const Node* root) {
7      std::vector<int> output;
8      if (root == nullptr) {
9          return output;
10     }
11     std::stack<const Node*> pending;
12     pending.push(root);
13     while (!pending.empty()) {
14         const Node* current = pending.top();
15         pending.pop();
16         output.push_back(current->value);
17         if (current->right != nullptr) {
18             pending.push(current->right.get());
19         }
20         if (current->left != nullptr) {
21             pending.push(current->left.get());
22         }
23     }
24     return output;
25 }
26
27 inline std::vector<int> inorder_iterative(const Node* root) {
28     std::vector<int> output;

```



```

29  std::stack<const Node*> pending;
30  const Node* current = root;
31  while (current != nullptr || !pending.empty()) {
32      while (current != nullptr) {
33          pending.push(current);
34          current = current->left.get();
35      }
36      current = pending.top();
37      pending.pop();
38      output.push_back(current->value);
39      current = current->right.get();
40  }
41  return output;
42 }
43
44 inline std::vector<int> level_order(const Node* root) {
45     std::vector<int> output;
46     if (root == nullptr) {
47         return output;
48     }
49     std::queue<const Node*> pending;
50     pending.push(root);
51     while (!pending.empty()) {
52         const Node* current = pending.front();
53         pending.pop();
54         output.push_back(current->value);
55         if (current->left != nullptr) {

```

代码清单 2: 迭代遍历与层序 frontier

显式结构保存的不是“所有节点”，而是尚未展开的边界；栈的压入顺序和队列的端点顺序直接生成输出次序。

8.3 Huffman 合并调用

```

1      return 0;
2  }
3  return 1 + std::max(height(root->left.get()), height(root->right.get()));
4  }
5
6  inline int huffman_weighted_path_length(const std::vector<int>& weights) {
7      if (weights.size() < 2) {
8          return 0;
9      }
10     std::priority_queue<int, std::vector<int>, std::greater<int>> pending(
11         weights.begin(), weights.end());
12     int total = 0;
13     while (pending.size() > 1) {
14         const int first = pending.top();
15         pending.pop();
16         const int second = pending.top();
17         pending.pop();

```

```

18     const int merged = first + second;
19     total += merged;
20     pending.push(merged);

```

代码清单 3: 最小权值合并与累计代价

8.4 测试证据

```

1  std::unique_ptr<Node> sample_tree() {
2      auto root = leaf(1);
3      root->left = leaf(2);
4      root->right = leaf(3);
5      root->left->left = leaf(4);
6      root->left->right = leaf(5);
7      return root;
8  }
9
10 void test_empty_and_singleton() {
11     std::vector<int> output;
12     preorder_recursive(nullptr, output);
13     assert(output.empty());
14     assert(level_order(nullptr).empty());
15     auto root = leaf(9);
16     assert(height(root.get()) == 1);
17     assert((preorder_iterative(root.get()) == std::vector<int>{9}));
18     assert((inorder_iterative(root.get()) == std::vector<int>{9}));
19 }
20
21 void test_traversal_orders() {
22     auto root = sample_tree();
23     std::vector<int> preorder;
24     std::vector<int> inorder;
25     std::vector<int> postorder;
26     preorder_recursive(root.get(), preorder);
27     inorder_recursive(root.get(), inorder);
28     postorder_recursive(root.get(), postorder);
29     assert((preorder == std::vector<int>{1, 2, 4, 5, 3}));
30     assert((preorder_iterative(root.get()) == preorder));
31     assert((inorder == std::vector<int>{4, 2, 5, 1, 3}));
32     assert((inorder_iterative(root.get()) == inorder));
33     assert((postorder == std::vector<int>{4, 5, 2, 3, 1}));
34     assert((level_order(root.get()) == std::vector<int>{1, 2, 3, 4, 5}));
35     assert(height(root.get()) == 3);
36 }

```

代码清单 4: 空树、遍历覆盖与 Huffman 边界测试

```

1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \
2     code/ds04_tree_binary_test.cpp -o /tmp/ds04_test
3 /tmp/ds04_test

```

9 复杂度与概念边界

操作	递归/迭代遍历	额外空间	前提	边界
遍历 n 节点	$O(n)$	$O(h)$ 或 $O(w)$	每节点一次	空树返回空序列
高度	$O(n)$	$O(h)$	子树高度定义固定	空树高度约定
Huffman 合并	$O(k \log k)$	$O(k)$	最小优先队列	$k < 2$ 代价为 0

概念 A	≠ 概念 B	判据	错误后果
树节点所有权	≠ 遍历顺序	生命周期 vs 访问时机	泄漏或次序错
层序遍历	≠ BFS 图遍历	树无环且无需一般 visited	把图结论硬套树
Huffman 树	≠ BST	最小带权路径 vs 有序查找	混淆目标

10 遇到题目时从哪里开始

题面出现树的访问顺序时，先判断是在**从树生成序列**还是**从序列恢复结构**。前者回到 DFS/BFS 与“根的记录时机”；后者回到“当前根 + 左右边界 + 是否存在额外形状约束”。完整纸面重建、唯一性/歧义计数、遍历序列相等或逆序速判转入《二叉树遍历与序列重建》；线索树的前驱/后继与空链域转入《线索二叉树的前驱后继》；树森林转换的 L/R 路径翻译与计数转入《树森林与孩子兄弟转换》；度数、层容量与指针槽位转入《树的计数与存储账本》。这些训练文档只拥有考场控制，不复制本册机制。

若题目问 Huffman，先确认是二叉还是 m 叉：二叉每轮取两个最小值； m 叉先判断是否需要补零，或等价地确定第一轮应该合并几个真实权值，之后每轮固定合并 m 个最小值。若题目给具体码字，就先查前缀冲突；若给最大码长，就按 Kraft/最深层位置计算剩余码空间。多叉树的每条边若又用固定长度 bit 串表示，还要先分清“树深”与“实际 bit 码长”。完整计算转入《Huffman 编码与前缀码》。

如果忘了具体口诀，不要硬背一个总口号。普通树问题先问：当前表示到底保存的是孩子、兄弟还是遍历邻接？这一步沿哪种关系走？算完能用什么独立条件验算？Huffman/编码问题则先问：长度按树边还是 bit 计？这一轮合并哪些最小权值？给出的码字先满足前缀条件吗，树的分支数又是否符合题设？

全科交接

DS04 从 DS02 接收 LIFO/FIFO frontier，从 DS05 接收最小优先队列接口；向 DS-B01 提供树遍历实例，向 DS09 提供“树结构可承载有序索引”的前置直觉，但不拥有 BST/AVL 机制。

旧总册关于递归层次、遍历访问时机与 Huffman 合并的内容已吸收；树/森林转换、线索化、遍历序列重建、带空标记序列化，以及“后序递归向父节点返回必要信息”的方法，也已按各自边界补入本册。更复杂的压缩编码和持久化格式仍只作 Extension，不能覆盖本册主干。